CITI BIKE REBALANCING EXPLORER · DOCUMENTATION SET V2

Document 02 — Data, Methods, Modeling, EDA, Findings & Limitations

The engine room

Datasets, data wrangling, feature engineering, the forecasting model and its honest result, the supporting analyses, what was found, what the tool cannot do, and where it goes next.

Assumes: Document 01 (the problem framing and the net-flow signal in one paragraph).

Baseline-first rule: plain, auditable methods first; a complex model is adopted only where it wins out-of-sample.

Prepared: July 2026 · every figure reproduced from committed data by code in `pipeline/`.

1. The pipeline, end to end

The analysis is a chain of small, single-purpose modules run in dependency order. `pipeline/reproduce_all.py` runs them as a human would at the command line and stops on the first real failure — it does not re-import internals, so "it reproduces" means the actual steps ran, not that a wrapper claimed success.

1. **Download & aggregate** (`build_full_year.py`) — pull 12 months of real Citi Bike trip archives; run QC; aggregate to net flow and a per-date daily table.
2. **Station typology** (`station_typology.py`) — k-means on flow *shape* to give each station a role label.
3. **Equity join** (`equity_join.py`) — nearest-distance from each station to NYCHA, schools, subway entrances.
4. **Weather elasticities** (`elasticities.py`) — how flow magnitude responds to temperature, precipitation, and capacity.
5. **Demand model** (`demand_model.py`) — the 12-fold walk-forward comparison of three forecasting tiers.
6. **Route & fleet** (`plan_routes.py`, `fleet_simulator.py`) — greedy overnight truck tours and a 1–10-truck sweep.
7. **Weather presets** (`scenario_presets.py`) — the named weather scenarios the dashboard offers.
8. **Ground-truth spot checks** (`spot_check.py`) — assert the output against facts a reviewer can verify without this repo.

2. The datasets

Dataset	Source & ID	Records	Used for
Citi Bike trips	citibikenyc.com system-data (S3)	12 months	Net flow, daily table, features
GBFS live status & info	gbfs.citibikenyc.com	live	Live dock layer; station capacity
NYCHA developments	data.cityofnewyork.us · `phvi-damg`	216	Equity proximity (≤ 300 m)
DOE school locations	data.cityofnewyork.us · `wg9x-4ke6`	1,899	Equity proximity (≤ 300 m)
MTA subway entrances	data.ny.gov · `i9wp-a4ja`	2,120	Transit gap (≥ 800 m)
Open-Meteo daily weather	archive-api.open-meteo.com	18 zones	Model features; elasticities

Record counts are from the committed `flows.json` equity-join block and data manifest. All sources are public and key-free.

> **COMMITTED SCALE**
>
> July 2025 – June 2026 (a full contiguous year) · **2,516** stations in the roster (**1,672** with full-year coverage) · the daily table underlying the weather work holds **788,979** real per-(station, date) rows.

3. Data wrangling & quality control

QC rules are fixed and documented in code (`pipeline/qc.py`). They are deliberately conservative and few, so they are auditable:

- **Duration bounds.** Trips under 60 s are dropped (false starts — undock/redock with no real ride); trips over 4 h are dropped (a bike never cleanly redocked).
- **Missing station IDs are kept and flagged, not dropped.** A trip missing only its end station still counts as a valid departure, and vice-versa. Because e-bikes can start or end outside the formal station network, this is common — so station-level departures and arrivals do *not* balance by construction. That is expected, not a bug.
- **Coordinates.** Station location is the *median* of its observed coordinates, to absorb GPS noise.

> **A REAL CONTAMINATION FIND, NOT A HYPOTHETICAL**
>
> While designing the multi-point weather step, three stations turned up in Citi Bike's own published S3 archive that do not belong to NYC: two named `"LA Metro Demo 2/3"` at real Los Angeles coordinates (34.03, −118.25), and one (`"Bronx WH station"`, SYS018) at (0, 0) — off the coast of Africa. All three had near-zero ridership but still inflated the station roster and the subway-gap flag count. A generous NYC bounding box (lat 39.5–41.5, lng −75 to −73) now drops any trip whose start or end falls outside it. This looks like internal warehouse/demo records leaking into the public export — real upstream contamination, caught by looking, not assumed away.

4. From trips to the net-flow signal

For each *(station, month, day-type, hour)* bucket: `net = arrivals − departures`, then divided by the number of distinct calendar dates observed in that bucket to give a **per-day rate** (a month has far more weekdays than weekend days, so raw counts are not comparable across day types). Month curves roll up into seasons — **winter** Dec–Feb, **spring** Mar–May, **summer** Jun–Aug, **fall** Sep–Nov — and an all-period average. The number the dashboard shows for a given hour is that period's value; it carries a per-day unit even though it describes a single hour (the "/day" normalizes across dates, not across hours).

5. Feature engineering

The daily table (`daily_net_flow.parquet`) carries one row per (station, date, hour) so that *every row has its own real day's weather*, not a monthly average — a fix that mattered (Section 8). Features added for the model:

- **Calendar:** hour, day-of-week, month, and U.S. federal holidays (via the `holidays` package).
- **Weather:** real daily temperature and precipitation, joined per date from the station's own weather zone (Section 7).
- **Station baseline:** the station's own trailing mean flow — a strong, cheap feature.
- **Typology label:** the station's cluster (Section 6), refit *inside each fold's training data only* so a held-out month never influences the label used to predict it.

6. Station typology — clustering on rhythm, not volume

Each station's 24-hour weekday curve is **L2-normalized to a unit vector before clustering**, so k-means groups stations by the *shape* of their day (drains AM / fills PM vs. the reverse), not by how busy they are. Clustering raw curves would let Euclidean distance be dominated by volume, answering "how busy" instead of "when does this station drain vs. fill."

k is chosen by silhouette score, not the elbow method — inertia falls monotonically as k grows (trivially minimized at k = n), while silhouette rewards separation from the next-nearest cluster, giving a real interior maximum to search:

k	2	3	4	5	6	7	8
silhouette	0.385	0.314	0.187	0.140	0.160	0.155	0.141

k = 2 is the clear maximum. Values from `flows.json` → `typology.silhouette_by_k`.

Cluster	Stations	Signature
Commuter core	684	Fills in the AM, drains in the PM (office / job-center stations)
Residential feeder	1,146	Drains in the AM, fills in the PM (where the commute starts)
Low signal (excluded)	686	Curve too flat/noisy to normalize into a real shape → cluster −1

1,830 stations clustered; 686 of 2,516 excluded as low-signal (raw weekday L2 norm < 1.0 bikes/day). Cluster centers are themselves 24-hour curves, so the taxonomy is auditable by inspection.

7. Multi-point weather — and an equity lesson in the method itself

Weather is read from **18 fixed geographic zones** (a **6 km grid** over the real station footprint), each fetched independently from Open-Meteo, so a station uses its own zone's weather rather than one city-wide reading.

> **WHY A GRID, NOT K-MEANS**
>
> The zones were originally k-means on station coordinates. Checked against the four equity-priority areas this project cares about most, it **skewed**: dense South Bronx and Upper Manhattan landed ~2–3 km from their zone centroid, but sparser East Queens and Southern Brooklyn were swept into distant, station-dense zones 6–8 km away. Raising k to 8 did not fix it (East Queens still ~5 km off). K-means balances zone *population*, which pulls centroids toward dense clusters — exactly the wrong bias for equity-priority areas. A fixed geographic grid has no notion of station density, so it cannot be pulled off course by it. Same lesson, later reused when evaluating depot data (Document 04).

8. The forecasting model — and its honest result

The model exists to answer one question under the baseline-first rule: *can a real model beat a plain seasonal-naive forecast out-of-sample?* Three tiers are compared:

1. **Seasonal-naive** — tomorrow's hour-h flow is the trailing average for that (station, month, day-type, hour). Fully reproducible with pandas alone. This is the benchmark to beat.
2. **GAM** — a spline generalized additive model.

3. **Guarded GBM** — a tuned gradient-boosting model with an explicit *extrapolation guard*: once a prediction's temperature leaves the range the model was trained on, it stops trusting the GBM and falls back to the GAM, then (past a wider margin) to the naive baseline.

Two techniques make the comparison fair. **Validation is a 12-fold walk-forward** (leave-one-month-out over the full year), not a single split — a single split cannot tell "extrapolation failure" apart from "the model is just worse." And the GAM/GBM predict **arrivals and departures as two independent non-negative count regressions (Poisson loss)**, recombined as `predicted_net = arrivals − departures`; the naive baseline is not duplicated per target because averaging the two and subtracting is mathematically identical to averaging net directly (verified by a dedicated test, not asserted).

RESULT — 12-FOLD WALK-FORWARD, MEAN MAE (BIKES/DAY), LOWER IS BETTER

Tier	Mean MAE	vs. naive
Seasonal-naive	1.954	— (benchmark)
GAM (splines)	2.180	worse · p = 0.0024
Guarded (GBM + GAM + naive)	2.096	worse · p = 0.012

The simple baseline wins, significantly, and it is not overturned. This is the honest headline. Fixing the earlier extrapolation problem (real per-date weather instead of monthly averages) did not flip the result — it just made the comparison statistically rigorous instead of a single lucky or unlucky split. Under the project's own rule, the complex model is *not* adopted, because it did not earn its complexity. The walk-forward table is shown in the dashboard so a reviewer sees this directly.

9. Weather & capacity elasticities (Investigator Mode Phase 1)

Separate from the forecast, the tool estimates how a station's *flow magnitude* responds to temperature, precipitation, and dock capacity — the numbers behind the weather-scenario projection. Temp/precip is a **direct joint linear regression** of real per-(station, date) daily flow magnitude on real daily temperature and precipitation; each coefficient carries a 95% confidence interval.

Typology group	Temp (per °C)	Precip (per mm)	Capacity	Daily obs.
Commuter core	+0.034	−0.017	+0.019	230,915
Residential feeder	+0.037	−0.018	+0.014	385,102

Unitless: fractional change in typical |net flow| per 1-unit change in the feature. Warmer ⇒ more flow; rain ⇒ less. All six point estimates have 95% CIs that exclude zero.

A CONFOUND FOUND AND CONTROLLED — BUT HONESTLY NOT MADE CAUSAL

A first capacity pass produced a *positive* elasticity in both groups — the opposite of the expected direction. Rather than ship it, the cause was checked: capacity and flow magnitude correlate at **r ≈ 0.68–0.74**, because DOT assigns busier stations more docks *and* busier stations independently swing more. Adding real ridership throughput as a third regression term and evaluating the capacity effect holding throughput fixed controls for that one confound. It does **not** make the estimate causal — it is still a cross-sectional comparison across different stations, not a "capacity changed at one station, here is what happened" measurement (which a public-data-only stack cannot observe). The output's own notes say so.

10. Route planner & fleet simulator

Overnight truck tours (`plan_routes.py`) are a greedy nearest-neighbor heuristic over stations flagged by their forecast AM net flow (hours 6–9). It is explicitly a prototype: the JSON contract is designed so an OR-Tools CVRP solver drops in without downstream changes. Two realism guards are written into the output so a reviewer sees them: a **depot assumption** (each truck starts at the largest remaining surplus station — a stopgap, since no public depot-location data exists) and a **45-stop per-truck cap** (a rough single-shift budget; without it, one truck once planned a 2,142-stop city-spanning tour).

FLEET FINDING — MARGINAL BENEFIT DOES *NOT* DIMINISH HERE

Of **1,037** AM-deficit and **615** AM-surplus flagged stations, 1 truck services 14 deficit / 9 surplus; 10 trucks service 145 deficit / 111 surplus — very close to **10×**, not a diminishing curve. Every truck hits its own 45-stop cap before running out of flagged stations, so the binding constraint is per-truck stop budget, not overall demand. A result worth stating precisely *because* it contradicts the intuitive "5th truck helps less than the 1st."

11. Equity join

Plain haversine nearest-neighbor from every station to each facility layer (brute force — the station × facility product is a few million operations, no spatial index needed). Thresholds are fixed: **≤ 300 m** to a NYCHA development or school flags proximity; **≥ 800 m** to the nearest subway entrance flags a transit gap. The raw distances (not just the flags) are written per station, so Investigator Mode can recompute counts live at any threshold.

> **A VINTAGE CAVEAT THAT TRAVELS WITH THE DATA**
>
> The school layer is the **2019–2020** DOE school-locations dataset — confirmed (re-checked 2026-07-23, not assumed) to be the only row-queryable, coordinate-bearing school dataset still served by NYC Open Data; the current layer is GIS-only and rejects row queries. Because this layer is the most likely of the three to have drifted, its vintage label is attached to the data and surfaced anywhere school-proximity numbers appear, not just noted here.

12. Verification & reproducibility

Two independent safety nets back every number:

- **Ground-truth spot checks** (`spot_check.py`) assert the output against facts a reviewer can verify without this repo — the Financial District behaves like an office district, Stuyvesant Town like a purely residential complex, Thanksgiving like a holiday, winter ridership below summer. None use the pipeline's own cluster labels (that would be circular). Failures are findings to report, never thresholds to quietly loosen.
- **One-command reproduction** (`reproduce_all.py`) re-derives every artifact from the committed data (`--skip-download`, the fast path) or from nothing but public data (full path, several GB).

> **ONE REPRODUCIBILITY CAVEAT, STATED PLAINLY**
>
> The live GBFS feed is a *current* snapshot with no historical archive, so capacity-derived numbers in `elasticities.json` drift slightly on every run by design. This was found and fixed once when the committed file turned out to have been built against a snapshot 9 days stale. Everything else — trip archives, Open-Meteo history, the fixed weather grid — reproduces identically. The automated test suite: **155 passed, 3 skipped.**

13. What the EDA and analyses actually showed

- **Strong seasonal amplitude.** January net-flow magnitude runs at roughly half of July's — the reason the whole tool slices by season/month rather than showing a year-round average.
- **Two dominant rhythms, cleanly separated.** Silhouette peaks sharply at k = 2: the city's stations divide into commuter-core (fills AM) and residential-feeder (drains AM), with a long tail of low-signal stations that genuinely have no strong rhythm.
- **Weather effects are real but modest and directionally sensible** (warm ⇒ more, rain ⇒ less), at a few percent per unit.
- **The simple forecast is hard to beat** (Section 8) — the single most important analytic finding, and the one the tool is built to display rather than bury.

- **An arrivals/departures split was tried and honestly lost.** Predicting the two sides separately and recombining did not beat predicting net directly for the naive baseline (it is an identity), and did not rescue the model tiers either — reported as a negative result, not hidden.

14. Limitations — stated plainly

- **Invisible demand.** Net flow cannot see the trips that did not happen because a station was empty or full. It is a demand-pressure proxy; the live GBFS layer is the complement.
- **No operator truck moves.** Trip data omits rebalancing-truck moves, so net flow measures demand pressure, not observed service failures. Joining the (non-public) truck logs is a day-one-on-the-job task.
- **Route planner is a greedy heuristic.** Realistic *within-budget* optimization is the OR-Tools CVRP upgrade path, not yet built.
- **Capacity elasticity is cross-sectional, not causal** (Section 9).
- **Weather scenarios are elasticity estimates, not a weather-specific model** — and the panel says so.
- **GBFS density.** The live-failure log behind a deferred "dock capacity sandbox" (Phase 5) has not yet accumulated enough real snapshots to be statistically useful.
- **School layer vintage** (Section 11).

15. Where it goes next

- **OR-Tools CVRP** router on the same JSON contract, replacing the greedy heuristic.
- **Per-station SARIMA with confidence intervals**, where interpretable intervals drive staffing.
- **E-bike vs. classic split** — a one-line groupby on the existing `rideable_type`.
- **Dock-capacity sandbox (Phase 5)** once the GBFS failure log reaches its density target.
- **Power BI port** — the flows table is deliberately tidy (station × day-type × hour) to drop into the DOT stack.
- **Truck-log join** to convert demand-pressure proxy into observed service performance.